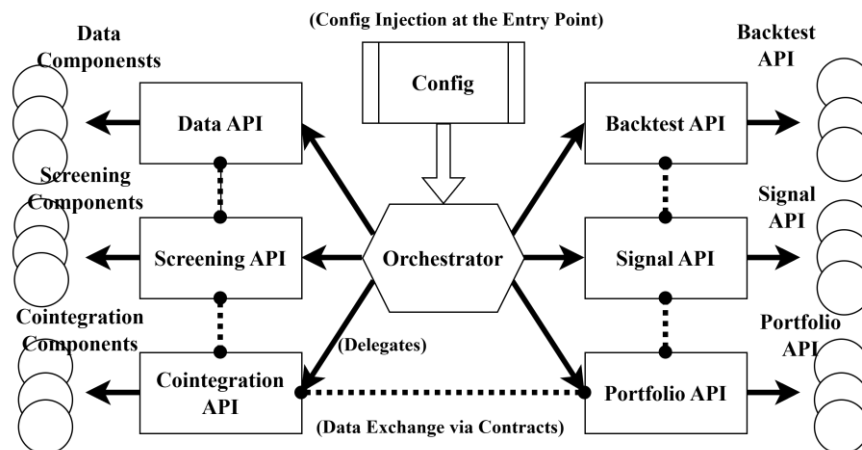


System Summary: Statistical Arbitrage in Japanese Equities | Takuya Kubo

1. System Objective

- Turn quantitative research ideas into executable trading experiments.
- Provide a reproducible framework for developing and evaluating statistical arbitrage strategies.
- Enable systematic comparison of alternative research methodologies.

2. Architecture



The Orchestrator coordinates domain-specific APIs, while each API encapsulates its own components and communicates through explicit Contracts.

3. Functions

Module		Responsibility	Key Components
Orchestrator		Pipeline orchestration	-
Config		Configuration management	-
Entry		System execution	-
Core	Data	Market data preparation	Universe / Loader
	Screening	Candidate generation	PCA / FF3 / Clustering
	Cointegration	Statistical evaluation	Johansen / Stationarity Evaluator
	Portfolio	Portfolio construction	Leverage / Allocator / Selector
	Signal	Signal generation	Kalman / Zscore
	Backtest	Trading simulation	Return Calculator
	Live	Live trading execution	IBKR
	Report	Report generation	Metrics / Export

4. Design Principle

Modularity

- Domain responsibilities are separated into independent modules.
- Research logic such as screening, cointegration analysis, and signal generation can be modified or replaced independently.
- New methodologies can be added without restructuring the entire pipeline.

Loose Coupling

- Components are isolated from one another.
- Each Component focuses on a specific task and does not coordinate with other Components.
- Coordination is handled by the Core API, while cross-module orchestration is handled by the Orchestrator.

Explicit Contracts

- Module boundaries are defined by explicit inputs and outputs.
- APIs exchange structured data through Contracts, rather than relying on implicit assumptions about internal implementations.
- This makes dependencies visible and interfaces stable.

Testability

- Components and APIs can be tested independently.
- Separation between orchestration, domain logic, and implementation allows failures to be isolated and research logic to be validated without running the entire system.

5. How to Run Backtest

Step 1: Configure

Set experiment parameters in **BacktestConfig**:

- Universe & date range
- Screening methodology
- Strategy parameters
- Other parameters

Step 2: Run

Run from the project root:

```
$ uv run python -m statarb
```

Step 3: Inspect Results

```
results/  
├── <experiment_id>/  
│   ├── config.csv  
│   ├── metrics.csv  
│   ├── cointegration.csv  
│   ├── portfolio.csv  
│   └── backtest.csv
```

Each experiment is stored in a dedicated result directory for reproducibility and comparison